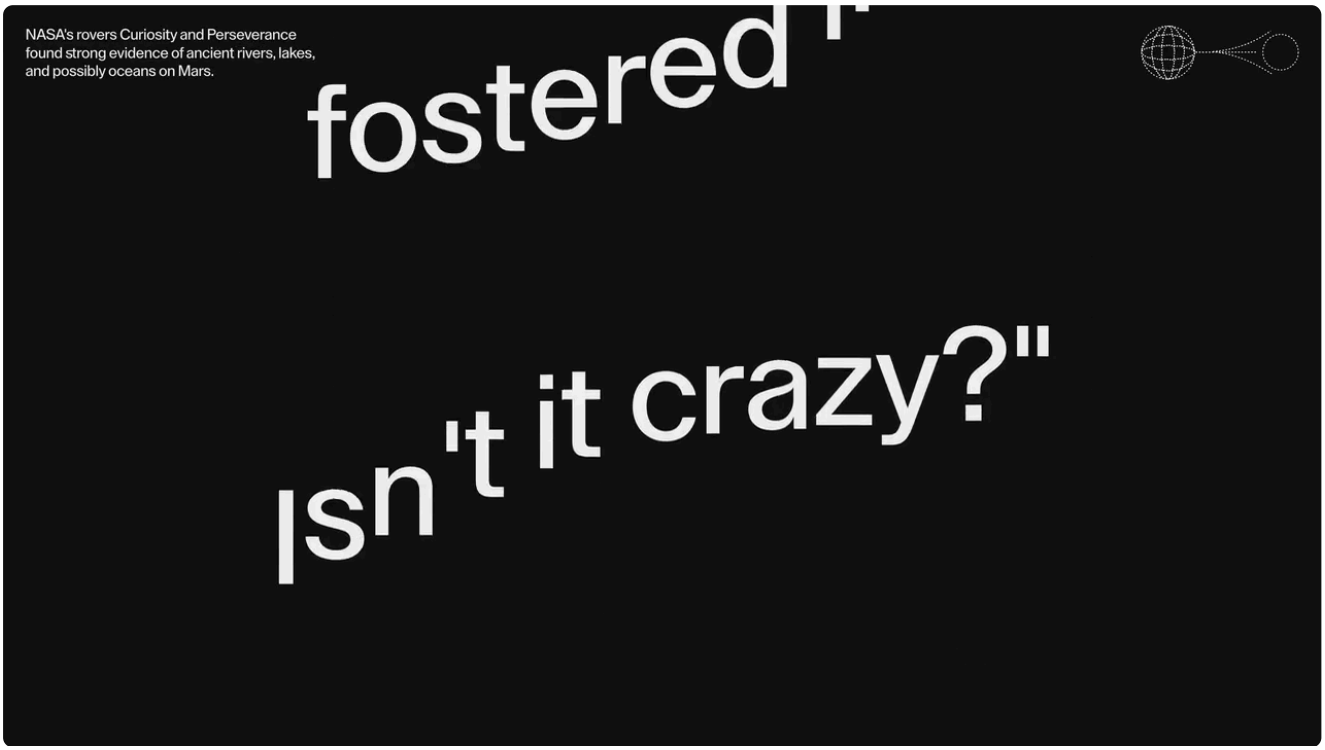
Made With Gsap

009
Up & Down



In this effect, we will display several short phrases letter by letter to keep the user's attention as they scroll through the page. The idea is to make the letters of one phrase disappear at the same time as the letters of the next phrase appear. Our logic will be dynamic: if we want to add one or more phrases, the effect will continue to work. Thanks a lot, JavaScript!

HTML Structure

The structure here is fairly simple: the `container` division holds the different phrases we will animate.

```
<div class="pin-height">
    <div class="container">
        <div class="center">
            <div class="sentence">"The Red Planet</div>
            <div class="sentence">may have once</div>
```

BATCH 001 — 050 EFFECTS

```
COPY/PASTE TO WEBFLOW AVAILABLE  SEE HOW EASY IT IS
Made With Gsap
    </div>
</div>
```

Some CSS

The `pin-height` division will set the height during which our `container` will remain pinned as the user scrolls. I chose a value of `600vh` for the scroll distance so that the effect fully plays out. If I reduce this value, the letters of the phrases will appear faster since there's less distance to scroll.

```
.mwg_effect009 .pin-height {
    height: 600vh;
}
.mwg_effect009 .container {
    display: flex;
    align-items: center;
    height: 100vh
}
```

Separating the letters of the phrases

In JavaScript, I start by applying a utility function to each phrase that will return a phrase with each letter wrapped in a `span` tag.

```
const root = document.querySelector('.mwg_effect009')
const sentences = root.querySelectorAll('.sentence')

sentences.forEach(sentence => {
    wrapLettersInSpan(sentence)
})
```

Let's look at the `wrapLettersInSpan()` method. I start by using the `split()` method, which divides the initial text into an array. Here, we use each character (`''`) as the separator. The `map()` method then adds the around each letter. Finally, I reconstruct

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS
Made With Gsap

is new string (with its DOM) is

parameter.

```
function wrapLettersInSpan(element) {
    const text = element.textContent;
    element.innerHTML = text
        .split('')
        .map(char => char === ' ' ? '<span> </span>' : `<span>${char}</span>`)
        .join(' ');
}
```

We notice the special case where the current character is a space: in that case, we assign to represent a non-breaking space.

Pinning the container

Next, I pin the `container` div to the full height of `pin-height` using an instance of `ScrollTrigger.create()`. This instance allows me to choose when to pin the container.

```
const pinHeight = root.querySelector('.pin-height')
const container = root.querySelector('.container')

ScrollTrigger.create({
    trigger: pinHeight, // Monitor the position of pin-height
    start: 'top top',
    end: 'bottom bottom',
    pin: container // The pinned section
})
```

To explain the code above: when the `pin-height` division reaches the top of the viewport, we start pinning the `container` until `pin-height` goes past the bottom of the viewport. This way, the `container` will remain perfectly visible on the screen throughout this distance.

Animating the letters

For the animation of the letters in each phrase, I declare a `gsap.timeline()`. This will play throughout the entire height of our `pinHeight` division.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

```
        start: 'top top',
        end: 'bottom bottom',
        scrub: true // Progress linked to scrolling
    }
})
```

All the tweens in the timeline will progress with the same parameters from the `scrollTrigger` object above. All the tweens will have the same duration, meaning each step in the timeline will be spread evenly over the entire distance of the animation.

Now we just need to make the letters of a sentence disappear while the letters of the next sentence appear. We will repeat this operation for each new sentence using the native `forEach()` function, targeting the current and next sentence using the `index` parameter.

Let's start by making the current sentence disappear from the center of the screen using the `gsap.to()` method. Each sentence's movement is divided into two parts: the sentence division moves up while its letters rise one by one.

```
sentences.forEach((sentence, index) => {
    // Move the sentence above the viewport using y & yPercent
    tl.to(sentences[index], {
        yPercent: -50,
        y: '-50vh',
        ease: 'power4.in',
    })

    // Move the letters above the sentence using y & yPercent
    tl.to(sentences[index].querySelectorAll('span'), {
        yPercent: -50,
        y: '-50vh',
        stagger: -0.02, // Stagger in the appearance
        ease: 'power2.in',
    }, '<') // Means the animation starts at the start of the previous tween
})
```

We notice the use of the `stagger: 0.2` attribute. But why 0.2? The important thing to know here is that the default `duration` for each tween is 0.5 seconds. Therefore, if each element animates for 0.5 seconds, and I stagger their start times by 0.5 seconds, that means they'd be perfectly sequenced. When we set `scrub: true` on a ScrollTriggered

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

een the start and end scroll

Made With Gsap

Thus, if we set `stagger` to 1 here (since `duration` is 1), a letter will appear exactly when the previous one finishes its movement. Reducing this value allows multiple letters to overlap at the same time.

Let's complete the `forEach()` function with two tweens for the next sentence apparition.
For these two tweens, I use the `gsap.from()` method. This method animates an element from a specified initial state to its current state.

```
sentences.forEach((sentence, index) => {
    ...

    // Move the next sentence (index+1)
    // to the middle of the viewport using y & yPercent
    tl.from(sentences[index+1], {
        yPercent: 50, // Starts at 50 and ends at 0
        y: '50vh', // Starts at 50vh and ends at 0
        ease: 'power4.out',
    }, '<')

    // Move the next letters (index+1)
    // to the middle of the viewport using y & yPercent
    tl.from(sentences[index+1].querySelectorAll('span'), {
        yPercent: 50, // Starts at 50 and ends at 0
        y: '50vh', // Starts at 50vh and ends at 0
        ease: 'power2.out',
        stagger: -0.02, // Stagger in the appearance
    }, '<')
})
```

In the current setup, the last sentence completely disappears at the end of the animation. Since the effect starts with the first sentence already visible, we will finish it by keeping the last sentence visible. To do this, we just need to add a condition that checks if a next sentence exists.
Here is our completed loop:

```
sentences.forEach((sentence, index) => {
    if(sentences[index+1]) { // Check is there is a sentence after
        // Move the sentence above the viewport using y & yPercent
        tl.to(sentences[index], {
            yPercent: -50,
            y: '-50vh',
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

```
        // Move the letters above the sentence using y & yPercent
        tl.to(sentences[index].querySelectorAll('span'), {
            yPercent: -50,
            y: '-50vh',
            stagger: -0.02, // Stagger in the appearance
            ease: 'power2.in',
        }, '<')

        // Move the next sentence (index+1)
        // to the middle of the viewport using y & yPercent
        tl.from(sentences[index+1], {
            yPercent: 50, // Starts at 50 and ends at 0
            y: '50vh', // Starts at 50vh and ends at 0
            ease: 'power4.out',
        }, '<')

        // Move the next letters (index+1)
        // to the middle of the viewport using y & yPercent
        tl.from(sentences[index+1].querySelectorAll('span'), {
            yPercent: 50, // Starts at 50 and ends at 0
            y: '50vh', // Starts at 50vh and ends at 0
            ease: 'power2.out',
            stagger: -0.02, // Stagger in the appearance
        }, '<')
    }
})
```

Go further

To take it further, we could have experimented with a different animation movement for the letters. How about scaling them up/down for the appearance and disappearance?

In this example, I used the Lenis smooth scroll to further enhance the fluidity and sliding effect. If you'd like to implement it as well, visit our Go Further page to learn how to easily add it.

Final code

HTML CSS JAVASCRIPT

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

```
            <p class="text">NASA's rovers Curiosity and Perseverance found strong evide
            <img class="visual" src="assets/medias/visual.svg" alt="">

            <div class="center">
                <div class="sentence">"The Red Planet</div>
                <div class="sentence">may have once</div>
                <div class="sentence">fostered life.</div>
                <div class="sentence">Isn't it crazy?"</div>
            </div>
        </div>
    </div>
</section>
```

Wanna learn something else?

BACK TO THE COLLECTION

Never miss what's next

Your email SUBMIT

By submitting your email, you'll be the first to know about upcoming updates for Made With Gsap. You can unsubscribe at any time.

SOCIAL

PAGES

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

FAQ

LINKEDIN

PRICING

CONTACT

REACH US

SUPPORT

Made With Gsap

2025©

PERMISSIONS & TERMS

PRIVACY POLICY